

5. First project

This section will guide you through creating your first RMF project using the instruction from the ROS2 Multirobot book - Traffic editor. We will follow the project structure of RMF demos, setting up necessary packages, and configuring dependencies and project settings in package.xml and CMakeLists.txt files. You will learn how to create a scenario configuration, define launch parameters, and set up fleet and robot configurations. Additionally, we will cover the setup of simulation launch files for testing your project in a simulated environment. By the end of this section, you will have a fully configured and functional RMF project ready for development and testing.

5.1. Create Project Structure and Basic Configuration

We will follow the project structure of RMF demos but will ignore some of the packages for now.

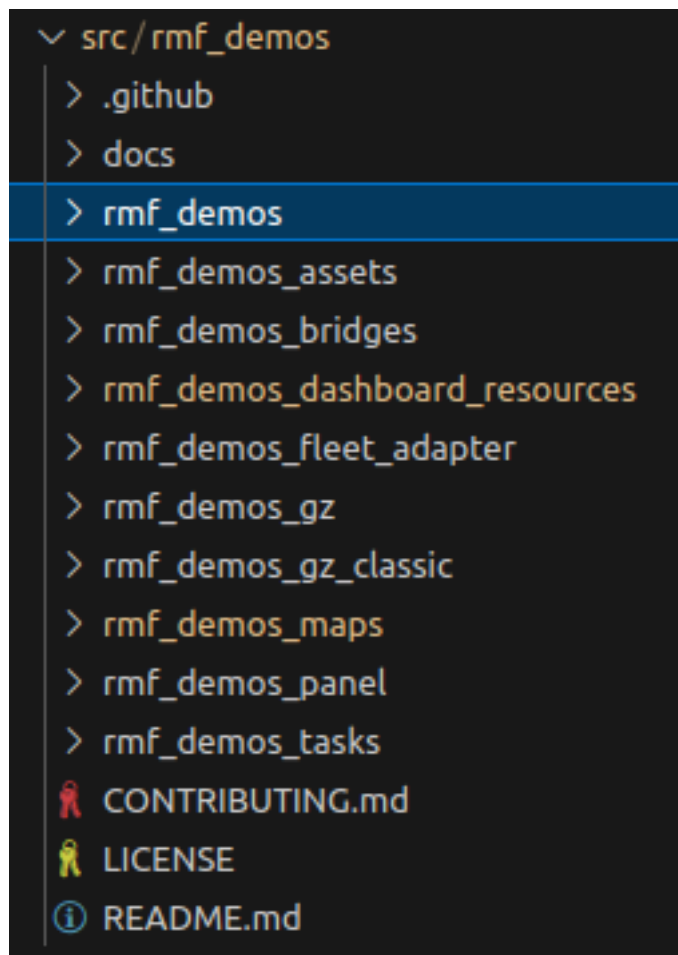


Figure 1: Project Structure

5.1.1. Copy and Create the Following Packages

1. Navigate to the `rmf-ws/src/` Directory and Create Your Project Directory:

```
$ cd rmf-ws/src/
```

```
$ mkdir project # change 'project' to your project name, we will use
                  'project' from now on
$ cd project
```

This creates a directory named `project` (replace `project` with your actual project name).

2. Copy the Necessary Assets and Create New ROS 2 Packages:

```
$ cp -r ../rmf_demos/rmf_demos_assets project_assets
$ ros2 pkg create project_config --license Apache-2.0
$ cp -r ../rmf_demos/rmf_demos_fleet_adapter project_fleet_adapter
$ ros2 pkg create project_maps --license Apache-2.0
$ ros2 pkg create project_simulation --license Apache-2.0
```

- `cp -r ../rmf_demos/rmf_demos_assets project_assets`: Copies the `rmf_demos_assets` directory from the RMF demos into your project, renaming it to `project_assets`.
- `ros2 pkg create project_config --license Apache-2.0`: Creates a new ROS 2 package named `project_config` with the specified license.
- `cp -r ../rmf_demos/rmf_demos_fleet_adapter project_fleet_adapter`: Copies the `rmf_demos_fleet_adapter` directory from the RMF demos into your project, renaming it to `project_fleet_adapter`.
- `ros2 pkg create project_maps --license Apache-2.0`: Creates a new ROS 2 package named `project_maps` with the specified license.
- `ros2 pkg create project_simulation --license Apache-2.0`: Creates a new ROS 2 package named `project_simulation` with the specified license.

3. Change the Name from `rmf_demos` to Your Project Name:

```
$ cd rmf-ws/src/project
$ find . -name 'rmf_demos*' -print -exec bash -c 'mv $0 ${0/rmf_demos
  /project}' {} \;
$ find . -name 'rmf_demos*' -print -exec bash -c 'mv $0 ${0/rmf_demos
  /project}' {} \;
$ find . -exec sed -i -e 's/rmf_demos/project/g' {} \;
```

- The first `find` command searches for files and directories named `rmf_demos*` and renames them to `project*`.
- The second `find` command repeats the renaming process to ensure all subfolders are correctly renamed.
- The third `find` command searches all files and replaces instances of `rmf_demos` with `project`.

5.1.2. Set Up Dependencies and Project Configuration

Next, configure the dependencies and project settings in each project's corresponding `package.xml` and `CMakeLists.txt` files.

The `package.xml` file contains information about the package such as name, version, maintainer, etc. It is important to pay attention to the `buildtool_depend` and `exec_depend` fields, where the former specifies the tooling required to build the project, while the latter specifies the dependencies of the package.

1. Edit `project_config/package.xml`:

```
<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd"
  schematypens="http://www.w3.org/2001/XMLSchema"?>
<package format="3">
  <name>project_config </name>
  <version>0.0.0</version>
  <description>Common launch files </description>
  <maintainer email="mail">email</maintainer>
  <license>Apache License 2.0</license>

  <buildtool_depend>ament_cmake</buildtool_depend>

  <exec_depend>project_maps </exec_depend>
  <exec_depend>rmf_traffic_ros2 </exec_depend>
  <exec_depend>rmf_task_ros2 </exec_depend>
  <exec_depend>rmf_fleet_adapter </exec_depend>
  <exec_depend>rmf_building_map_tools </exec_depend>
  <exec_depend>rmf_visualization </exec_depend>
  <exec_depend>project_assets </exec_depend>
  <exec_depend>project_tasks </exec_depend>
  <exec_depend>project_panel </exec_depend>
  <exec_depend>project_fleet_adapter </exec_depend>
  <exec_depend>rviz2 </exec_depend>
  <exec_depend>launch_xml </exec_depend>

  <export>
    <build_type>ament_cmake</build_type>
  </export>
</package>
```

- `<name>project_config</name>`: Specifies the name of the package.

- `<version>0.0.0</version>`: Specifies the version of the package.
- `<maintainer email="mail">email</maintainer>`: Specifies the maintainer's email.
- `<buildtool_depend>ament_cmake</buildtool_depend>`: Specifies `ament_cmake` as a build tool dependency.
- `<exec_depend>...`: Specifies the runtime dependencies for the package.

The `CMakeLists.txt` file contains the build instructions for the package.

1. Edit `project_config/CMakeLists.txt`:

```
cmake_minimum_required(VERSION 3.8)
project( project_config )

if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
  add_compile_options(-Wall -Wextra -Wpedantic)
endif()

# find dependencies
find_package(ament_cmake REQUIRED)
# uncomment the following section in order to fill in
# further dependencies manually.
# find_package(<dependency> REQUIRED)

install(DIRECTORY
  launch /
  config /
  DESTINATION share/${PROJECT_NAME}
)

ament_package()
```

- `cmake_minimum_required(VERSION 3.8)`: Sets the minimum required version of CMake.
- `project( project_config )`: Sets the project name.
- `if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")...`: Adds compiler options for GCC or Clang.
- `find_package(ament_cmake REQUIRED)`: Finds the `ament_cmake` package which is required for building.
- `install(DIRECTORY launch/ config/ DESTINATION share/${PROJECT_NAME})`: Installs the `launch` and `config` directories to the specified destination.
- `ament_package()`: This macro must be called for any package to be recognized by ROS 2's build system.

By following these steps, you will set up the basic structure and configuration for your project based on the RMF demos. Ensure that you adjust the `package.xml` and `CMakeLists.txt` files according to your project's specific requirements and dependencies.

5.2. Configure scenario

To start using the Robotics Middleware Framework (RMF), you need to create a scenario, which involves setting up the configuration and launch files in `ros2` launch file formats necessary for the RMF to operate. This guide will help you create your first scenario, named 'test1'.

RMF configuration is extensive and complex, it may take some time to understand what anything does. The first section of these files are the arguments, if no value is set and no default value is provided it throws an error. The other part works by composing different `launch.xml` files that reference to one another, this is where you can set the values of the arguments, think of them as calls to other modules.

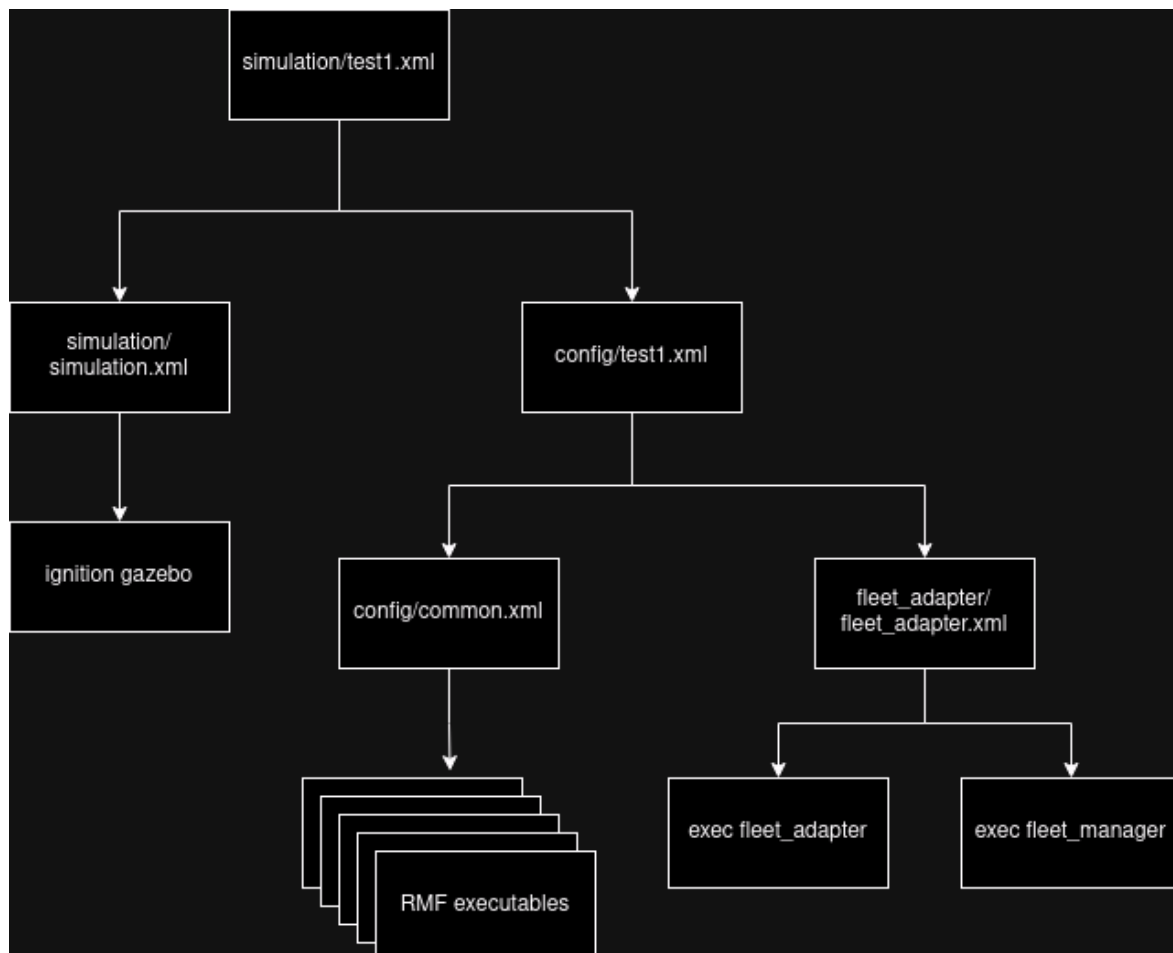


Figure 2: Launch files composition structure

In the configuration package (`project_config`), you need to add two folders: `config` and `launch`. Similarly, in

the simulation package (project_simulation), you need to add a launch folder. These folders will contain the necessary files to set up and run the scenario.

5.2.1. Project config

project_config/launch/common.launch.xml

This file launches the main functionality of RMF, such as the traffic scheduler, supervisors, and dispatchers. Below is an explanation of its structure:

```
<?xml version = '1.0' ?>

<launch>

  <arg name="use_sim_time" default="false" description="Use the /clock
    topic for time to sync with simulation"/>
  <arg name="viz_config_file" default="$(find -pkg-share
    rmf_visualization_schedule)/config/rmf.rviz"/>
  <arg name="config_file" description="Building description file required
    by rmf_building_map_tools"/>
  <arg name="dashboard_config_file" default="" description="Path to
    dashboard config for web rmf panel file"/>
  <arg name="initial_map" default="L1" description="Initial map name for
    the visualizer"/>
  <arg name="headless" default="false" description="do not launch rviz;
    launch gazebo in headless mode"/>
  <arg name="bidding_time_window" description="Time window in seconds for
    task bidding process" default="2.0"/>
  <arg name="use_rmf_panel" default="true" description="launch web and
    api server for rmf_demos_panel"/>

  <!-- Traffic Schedule -->
  <node pkg="rmf_traffic_ros2" exec="rmf_traffic_schedule" output="both"
    name="rmf_traffic_schedule_primary">
    <param name="use_sim_time" value="$(var use_sim_time)"/>
  </node>

  <!-- Blockade Moderator -->
  <node pkg="rmf_traffic_ros2" exec="rmf_traffic_blockade" output="both">
    <param name="use_sim_time" value="$(var use_sim_time)"/>
  </node>
```

```

<!-- Building Map -->
<group>
  <node pkg="rmf_building_map_tools" exec="building_map_server" args="$
    (var config_file)">
    <param name="use_sim_time" value="$(var use_sim_time)"/>
  </node>
</group>

<!-- Visualizer -->
<group>
  <include file="$(find-rmf-share rmf_visualization)/visualization.
    launch.xml">
    <arg name="use_sim_time" value="$(var use_sim_time)"/>
    <arg name="map_name" value="$(var initial_map)"/>
    <arg name="viz_config_file" value="$(var viz_config_file)"/>
    <arg name="headless" value="$(var headless)"/>
  </include>
</group>

<!-- Door Supervisor -->
<group>
  <node pkg="rmf_fleet_adapter" exec="door_supervisor">
    <param name="use_sim_time" value="$(var use_sim_time)"/>
  </node>
</group>

<!-- Lift Supervisor -->
<group>
  <node pkg="rmf_fleet_adapter" exec="lift_supervisor">
    <param name="use_sim_time" value="$(var use_sim_time)"/>
  </node>
</group>

<!-- Dispatcher Node -->
<group>
  <node pkg="rmf_task_ros2" exec="rmf_task_dispatcher" output="screen
    ">
    <param name="use_sim_time" value="$(var use_sim_time)"/>

```

```

        <param name="bidding_time_window" value="$(var bidding_time_window)
        "/>
    </node>
</group>

<!-- <group if="$(var use_rmf_panel)">
    <include file="$(find-rmf-share rmf_demos)/dashboard.launch.xml">
        <arg name="use_sim_time" value="$(var use_sim_time)"/>
        <arg name="dashboard_config_file" value ="$(var
            dashboard_config_file)"/>
    </include>
</group> -->

</launch>

```

Explanation:

- **Arguments (<arg>):** These are configurable parameters for the launch file. For instance, `use_sim_time` determines if the simulation time is used and it is a very importante parameter, when start the simulation you decide to use sim time or not, any other execution to that simulation like a task or a new package must also use the same option in `use_sim_time` or it won't work.
- **Nodes (<node>):** These represent individual processes that are launched. Each node belongs to a package and executes a specific program. There some from RMF like traffic scheduler and blockade moderator that control the pathing system, others like building map tools that are used to parse the yaml file created with `traffic_editor` to a rmf-compatible file and other tools like supervisors to make door and lifts work or visualizer that launches rviz.
- **Groups (<group>):** These are used to logically group nodes and other launch configurations together.

project_config/launch/test1.launch.xml

This file defines scenario-specific launch parameters and executes the common launch. It essentially includes the common launch file and adds specific configurations for the 'test1' scenario.

```

<?xml version='1.0' ?>

<launch>
    <arg name="use_sim_time" default="false"/>

    <!-- Common launch -->
    <include file="$(find-rmf-share project_config)/common.launch.xml">
        <arg name="use_sim_time" value="$(var use_sim_time)"/>
        <arg name="viz_config_file" value ="$(find-rmf-share project_config)/

```



```

    include / test1 / test1 . rviz "/>
    <arg name="config_file" value="$(find -pkg-share project_maps) / test1 /
    test1 . building . yaml"/>
    <!-- <arg name="dashboard_config_file" value="$(find -pkg-share
    config_dashboard_resources) / test1 / dashboard_config . json"/> -->
</include>

<!-- Tinyrobot fleet adapter -->
<group>
    <include file="$(find -pkg-share project_fleet_adapter) / launch /
    fleet_adapter . launch . xml">
    <arg name="use_sim_time" value="$(var use_sim_time)"/>
    <arg name="nav_graph_file" value="$(find -pkg-share project_maps) /
    maps / test1 / nav_graphs / 1 . yaml" />
    <arg name="config_file" value="$(find -pkg-share project_config) /
    config / test1 / tinyRobot_config . yaml"/>
    </include>
</group>

</launch>

```

Explanation:

- This file uses the `<include>` tag to incorporate other launch files, effectively creating a modular launch setup that cascades into launching the whole system.
- Arguments specific to the scenario 'test1' are set here and passed onto the other launch files.
- Take special attention to 'nav_graph_file', this are the lanes the the robot can traverse in the map. It will be explained latter in the create map section.

project_config/config/test1/tinyRobot_config.yaml

This YAML file defines the configuration for the tiny robot fleet. It includes both fleet-level and robot-specific parameters:

```

# FLEET CONFIG
=====
# RMF Fleet parameters

rmf_fleet:
  name: "tinyRobot"
  fleet_manager:
    ip: "127.0.0.1"

```

```

    port: 22011
    user: "some_user"
    password: "some_password"
limits:
    linear: [0.5, 0.75] # velocity, acceleration
    angular: [0.6, 2.0] # velocity, acceleration
profile: # Robot profile is modelled as a circle
    footprint: 0.3 # radius in m
    vicinity: 0.5 # radius in m
reversible: True # whether robots in this fleet can reverse
battery_system:
    voltage: 12.0 # V
    capacity: 24.0 # Ahr
    charging_current: 5.0 # A
mechanical_system:
    mass: 20.0 # kg
    moment_of_inertia: 10.0 #kgm^2
    friction_coefficient: 0.22
ambient_system:
    power: 20.0 # W
tool_system:
    power: 0.0 # W
recharge_threshold: 0.10 # Battery level below which robots in this
    fleet will not operate
recharge_soc: 1.0 # Battery level to which robots in this fleet should
    be charged up to during recharging tasks
publish_fleet_state: 10.0 # Publish frequency for fleet state, ensure
    that it is same as robot_state_update_frequency
account_for_battery_drain: True
task_capabilities: # Specify the types of RMF Tasks that robots in this
    fleet are capable of performing
loop: True
delivery: True
clean: False
finishing_request: "park" # [park, charge, nothing]

# TinyRobot CONFIG
=====

```

```

robots :
  # Here the user is expected to append the configuration for each robot
  # in the
  # fleet .
  # Configuration for tinyRobot1
  tinyRobot1 :
    robot_config :
      max_delay: 15.0 # allowed seconds of delay of the current itinerary
      before it gets interrupted and replanned
    rmf_config :
      robot_state_update_frequency: 10.0
      start :
        map_name: "L1"
        waypoint: "tinyRobot1_charger"
        orientation: 0.0 # radians
      charger :
        waypoint: "tinyRobot1_charger"

```

Explanation:

- **Fleet Configuration (rmf_fleet):** Contains overall parameters for the fleet, such as server configuration, speed limits, battery system specifications, and task capabilities.
- **Robot Configuration (robots):** Details specific to each robot in the fleet, including their initial state and charging station.

5.2.2. Project simulation

project_simulation/launch/simulation.launch.xml

This file launches the Gazebo simulator, setting up the simulation environment:

```

<?xml version='1.0' ?>

<launch>
  <arg name="map_package" default="project_maps" description="Name of the
    map package" />
  <arg name="map_name" description="Name of the project map to simulate"
    />
  <arg name="use_crowdsim" default='0' />
  <arg name="use_tpe" default='0' />
  <arg name="ignition_version" default='6' />

```

```

<let name="world_path" value="$(find-pkg-share $(var map_package))/maps
    /$(var map_name)_ign/$(var map_name).world" />
<let name="model_path" value="$(find-pkg-share $(var map_package))/maps
    /$(var map_name)/models:$(find-pkg-share $(var map_package))/maps/$(
    var map_name)_ign/models:$(find-pkg-share project_assets)/models:$(
    env HOME)/.gazebo/models" />
<let name="plugin_path" value="$(find-pkg-prefix
    rmf_robot_sim_gz_plugins)/lib/rmf_robot_sim_gz_plugins:$(find-pkg-
    prefix rmf_building_sim_gz_plugins)/lib/rmf_building_sim_gz_plugins"
    />
<!-- Use TPE if `use_tpe` is set to true-->
<!-- TODO use the variable below and remove code duplication (second
    ign gazebo command) when substitution in XML with ignition gazebo
    works -->
<let name="tpe_engine" value="\-physics-engine ignition-physics-tpe-
    plugin" />
<let name="tpe_engine" unless="$(var use_tpe)" value="" />

<!-- Use crowd sim if `use_crowdsim` is true-->
<let name="menge_resource_path" if="$(var use_crowdsim)" value="$(find-
    pkg-share $(var map_package))/maps/$(var map_name)/config_resource
    "/>
<let name="menge_resource_path" unless="$(var use_crowdsim)" value=""
    />

<let name="ign_headless" if="$(var headless)" value="-s"/>
<let name="ign_headless" unless="$(var headless)" value="" />

<group unless="$(var use_tpe)">
    <executable cmd="ign gazebo --force-version $(var ignition_version) $
        (var ign_headless) $(var tpe_engine) -r -v 4 $(var world_path)"
        output="both">
        <env name="IGN_GAZEBO_RESOURCE_PATH" value="$(var model_path):$(var
            world_path)" />
        <env name="IGN_GAZEBO_SYSTEM_PLUGIN_PATH" value="$(var plugin_path)
            "/>
        <env name="IGN_GUI_PLUGIN_PATH" value="$(var plugin_path)"/>
        <env name="MENGE_RESOURCE_PATH" value="$(var menge_resource_path)
            "/>

```

```

    </executable>
</group>

<group if ="$(var use_tpe)">
  <executable cmd="ign gazebo $(var ign_headless) --physics-engine
    ignition-physics-tpe-plugin -r -v 4 $(var world_path)" output="
    both">
    <env name="IGN_GAZEBO_RESOURCE_PATH" value="$(var model_path):$(var
      world_path)" />
    <env name="IGN_GAZEBO_SYSTEM_PLUGIN_PATH" value="$(var plugin_path)
      "/>
    <env name="IGN_GUI_PLUGIN_PATH" value="$(var plugin_path)"/>
    <env name="MENGE_RESOURCE_PATH" value="$(var menge_resource_path)
      "/>
  </executable>
</group>

<!-- ros_ign bridge for simulation clock -->
<node pkg="ros_ign_bridge" exec="parameter_bridge"
  args="/clock@rosgraph_msgs/msg/Clock[ignition.msgs.Clock]"
  />

</launch>

```

Explanation:

- **Arguments (<arg>):** These set up various parameters for the simulation, such as which map to use(map_name), where to find it(map_package) and whether to use crowd simulation.
- **Variable (<let>):** These create different variables for the simulation and there are defined based on the arguments.
- **Executable (<executable>):** This runs the Gazebo simulation, with different configurations based on whether TPE (Time Parameterized Execution) is used.
- **Node (<node>):** This sets up the bridge between ROS and Gazebo, allowing ROS to control the Gazebo simulation.

project_simulation/launch/test1.launch.xml

This file is the main entry point, invoking the necessary launch files to run the ‘test1’ scenario:

```

<?xml version='1.0' ?>

<launch>

```

```

<arg name="use_sim_time" default="true"/>
<arg name="failover_mode" default="false"/>
<arg name="use_tpe" default="false"/>
<arg name="ignition_version" default='6'/>

<!-- Common launch -->
<include file="$(find-pkg-share project_config)/test1.launch.xml">
  <arg name="use_sim_time" value="$(var use_sim_time)"/>
  <arg name="failover_mode" value="$(var failover_mode)"/>
  <arg name="use_tpe" value="$(var use_tpe)"/>
</include>

<!-- Simulation launch -->
<include file="$(find-pkg-share project_simulation)/simulation.launch.xml">
  <arg name="map_name" value="test1" />
  <arg name="ignition_version" value="$(var ignition_version)" />
</include>

</launch>

```

Explanation:

- This file combines the common launch and simulation launch files to set up and run the ‘test1’ scenario.
- Arguments specific to the ‘test1’ scenario are set here, such as use_sim_time and failover_mode.

5.3. Create a Map

To create a map, we can use the ‘traffic-editor’ tool that comes with RMF, we will guide you through the process step by step with examples and you can also check the ROS2 traffic editor documentation. Execute `traffic – editor` in the terminal, go to Building → New, and create a new `test1 . building .yaml` file in `project_maps/maps/test1 /`. Select ‘reference image coordinates’.

1. Create a Level:

- Press the “Add...” button on the top-right side of the screen. Name the level L1 and press OK as shown in the Figure 3.

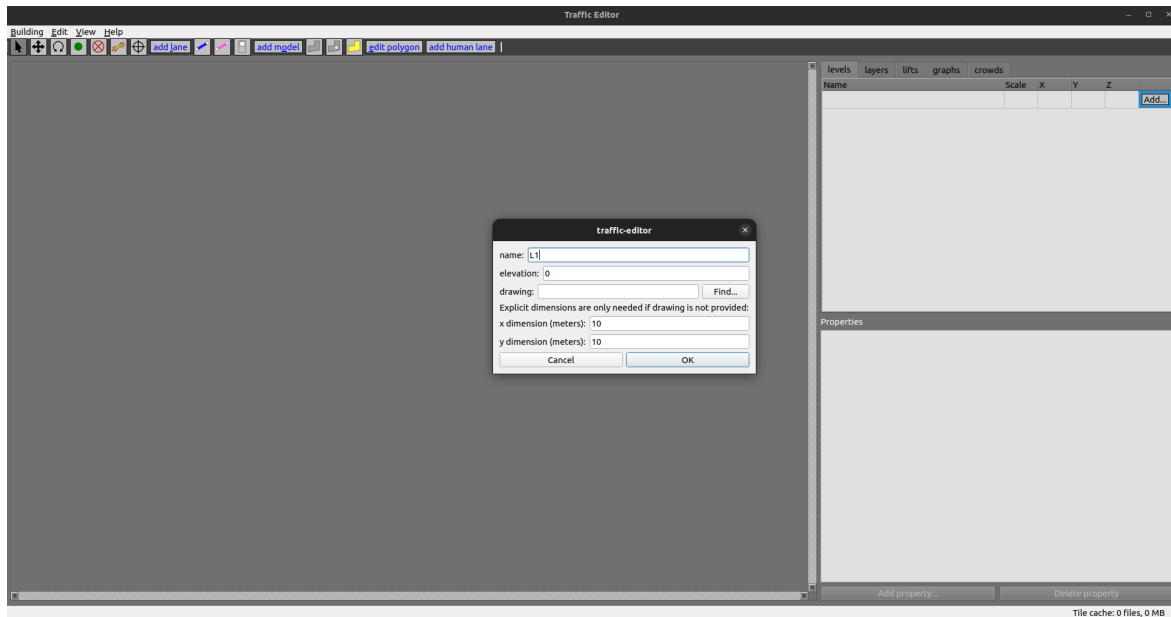


Figure 3: Creating a New Level

2. Add Walls:

- Press the wall button (marked as a blue line) or press 'W' to use the add wall tool. The Wall tool will add vertices for you and join them with walls (Figure 4). You can also set the vertices (green dot or press 'V') before the walls for more control over the layout. Vertices are key for the level layout as they can become walls, doors, floors, measurements, lanes, and more.

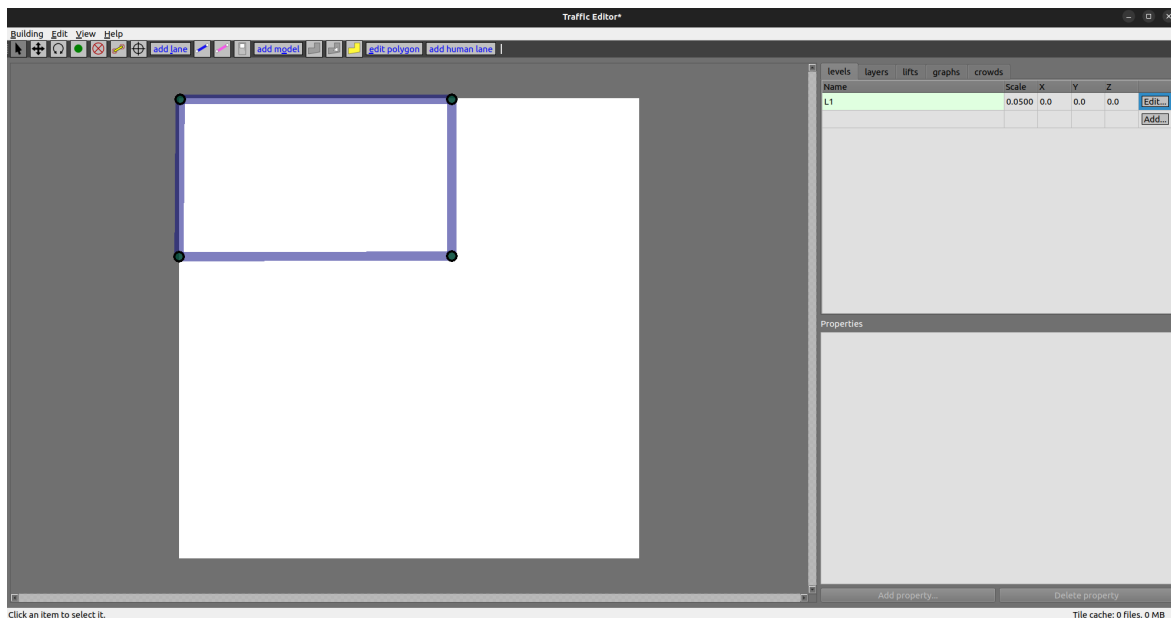


Figure 4: Adding Walls

3. Align Vertices:

- Since this process is manual there is always error in the placement so we need to fix the position of the vertices manually. Select the top-left vertex using the select tool (cursor button or press 'Esc') and check its properties on the bottom-right panel. You will notice the 'x (pixels)' and 'y (pixels)' values. Set both to 0 and align the other three vertices to create a rectangle with a size of 100x150 as reflected in Figure 5.

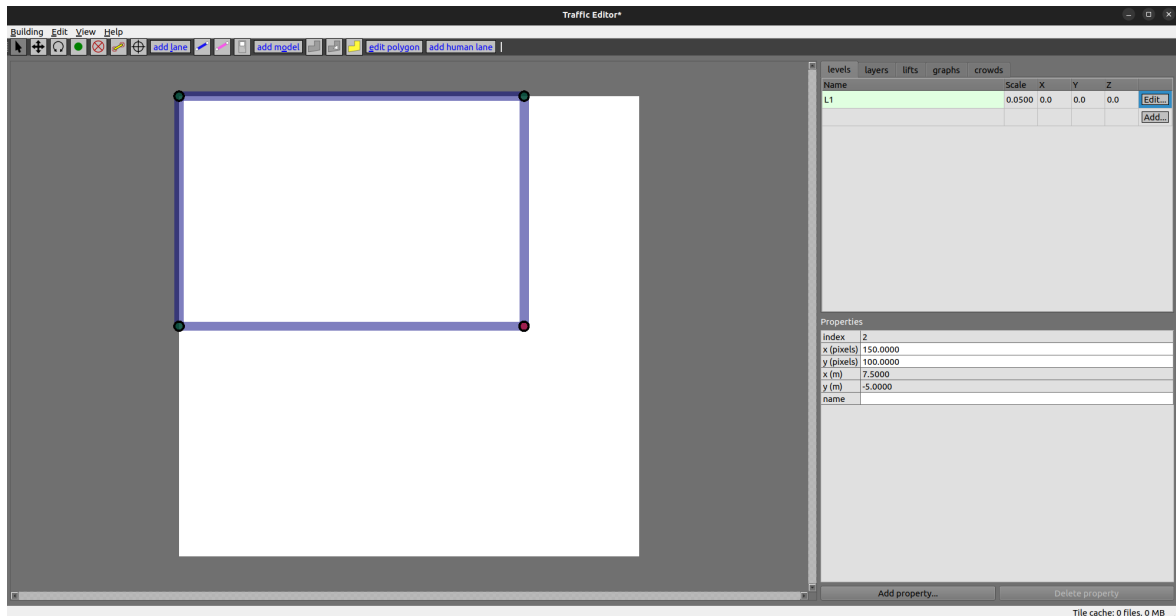


Figure 5: Aligning Walls

4. Add a Floor:

- Use the same vertices to add the floor with the floor tool (marked as a grey polygon or press 'F'), just click all the vertices in clock-wise or counter clock-wise order and right-click when done(Figure 6).

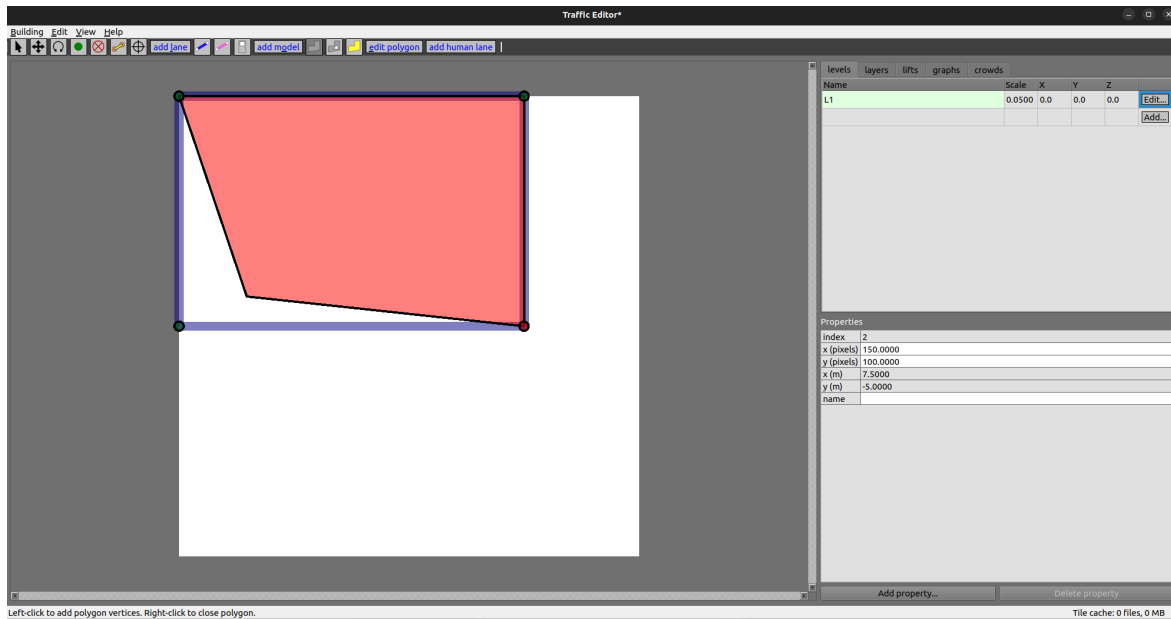


Figure 6: Adding a Floor

5. Spawn a Robot:

- Create a vertex at coordinates (50, 50), remember, use 'V' to apply the vertex tool. Set the x and y properties to 50 and add the spawn_robot_name, spawn_robot_type, and is_charger properties (Figure 7), this is done by clicking in the "Add property..." button and selection the proper options in the dropdown menu. Set the name of the vertex to tinyRobot1_charger, is_charger to true, spawn_robot_name to tinyRobot1 and spawn_robot_type to TinyRobot (Figure 8). Check the configuration file project_config / config / test1 / tinyRobot_config .yaml as this properties should have the same names.

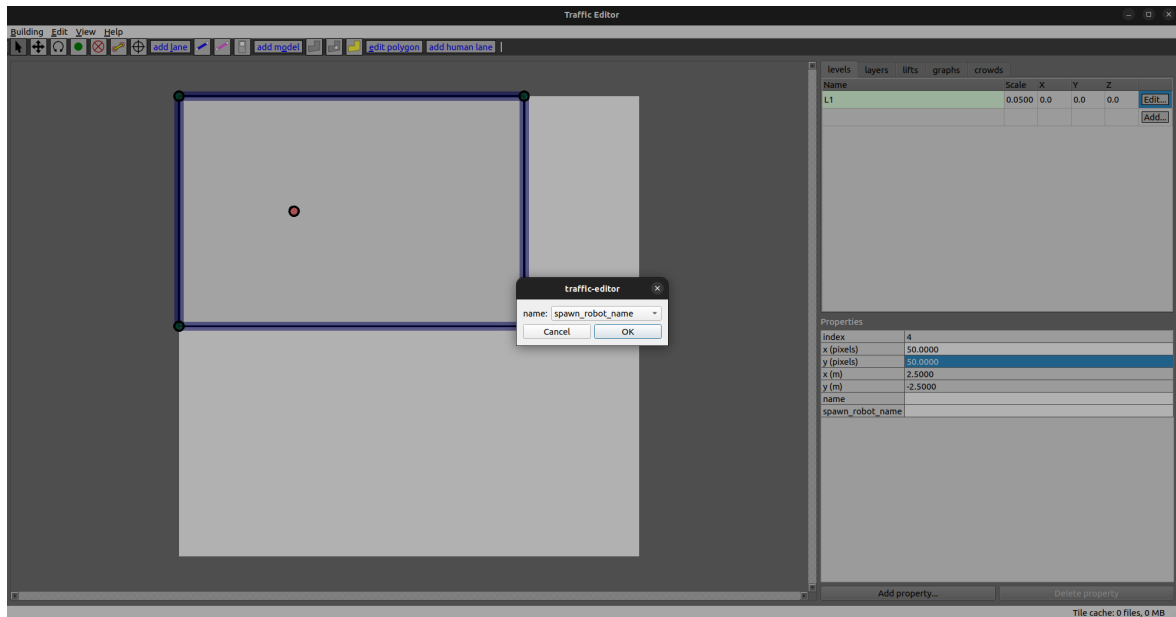


Figure 7: Spawning a Robot

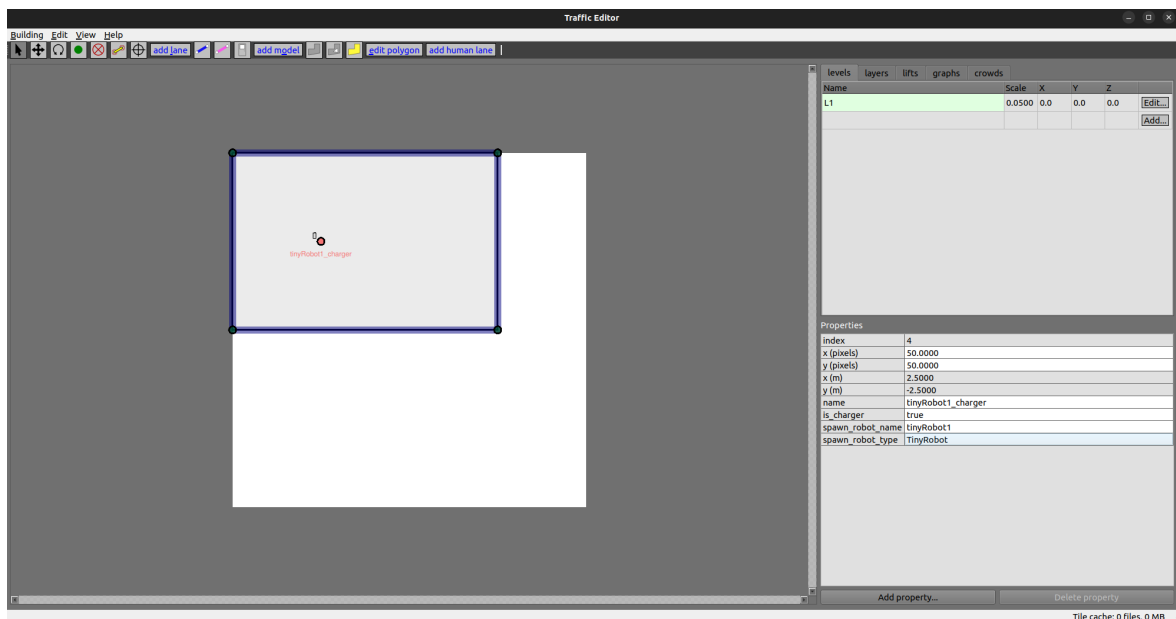


Figure 8: Robot Configuration

6. Add a Lane:

- To allow the robot to move, click on add_lane or press 'L' to invoke the lane tool. Select the vertex used for the robot spawn point and create a lane by clicking on another vertex or in any place in the map (this will create a new vertex). Press 'Esc' and if you created a new vertex select it, then change its x and y coordinates to (100, 50) and name it waypoint1 (Figure 9).

- It is possible to create different lane levels for different robots, by pressing the number from 0 to 9, this will assign a different color to the lanes you create. In this example we are focusing on just one lane, usually '0' by default, but in this case we will use '1' by pressing the number '1'. If you want to use the different navigation graphs that are created from this, you will need to add in each robot's yaml configuration the proper `nav_graph_file` property.

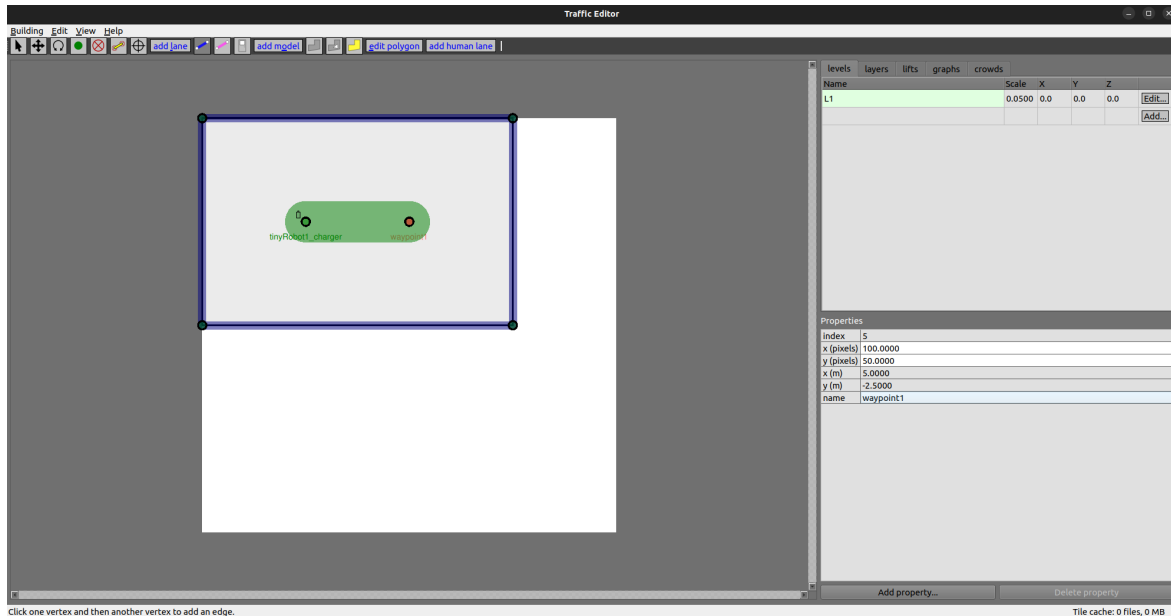


Figure 9: Adding a Lane

7. Run the Simulation:

- We configured a small room with a robot, let's check in the simulation. Use `colcon` to build the project, source the `setup.bash`, and launch the simulation with the following command, this will build the project and set up all dependencies for ROS to launch the simulation:

```
colcon build && source install/setup.bash && ros2 launch
  project_simulation test1.launch.xml
```

- As observed in Figure 10, two windows should pop up: Gazebo (with a 3D representation of the map) and RViz (initially empty but can be configured to show important information about lanes and robots).

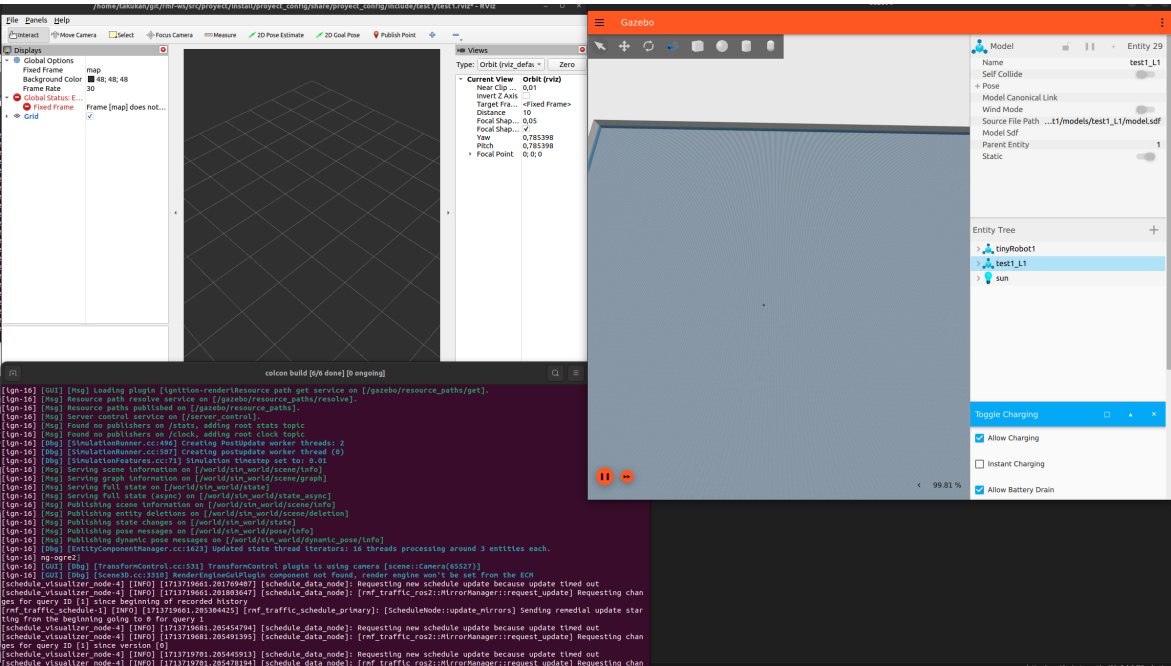


Figure 10: First Launch

8. Set Measurements:

- The robot appears tiny because no measurements have been set so the system uses one by default. Click on the pink line next to the wall or press 'T' to use the measurement tool. Click on two vertices and set the distance between them. Click on the measurement line and adjust the distance to a more comfortable size (e.g., 10 units). Usually, the layout is created using a blueprint of the building with proper measures.

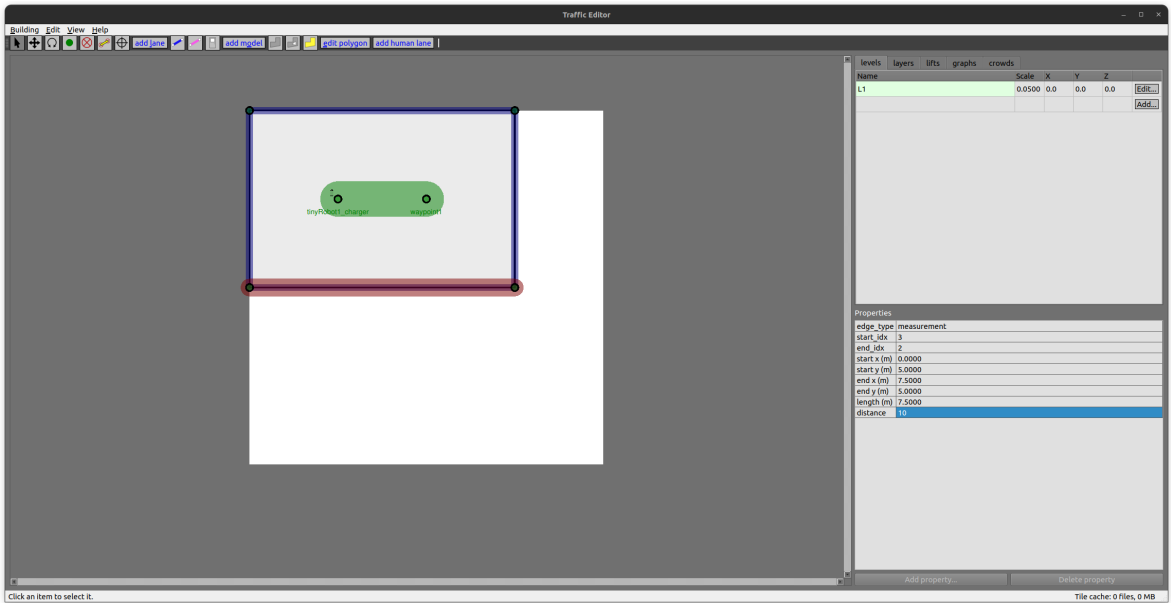


Figure 11: Setting Measurements

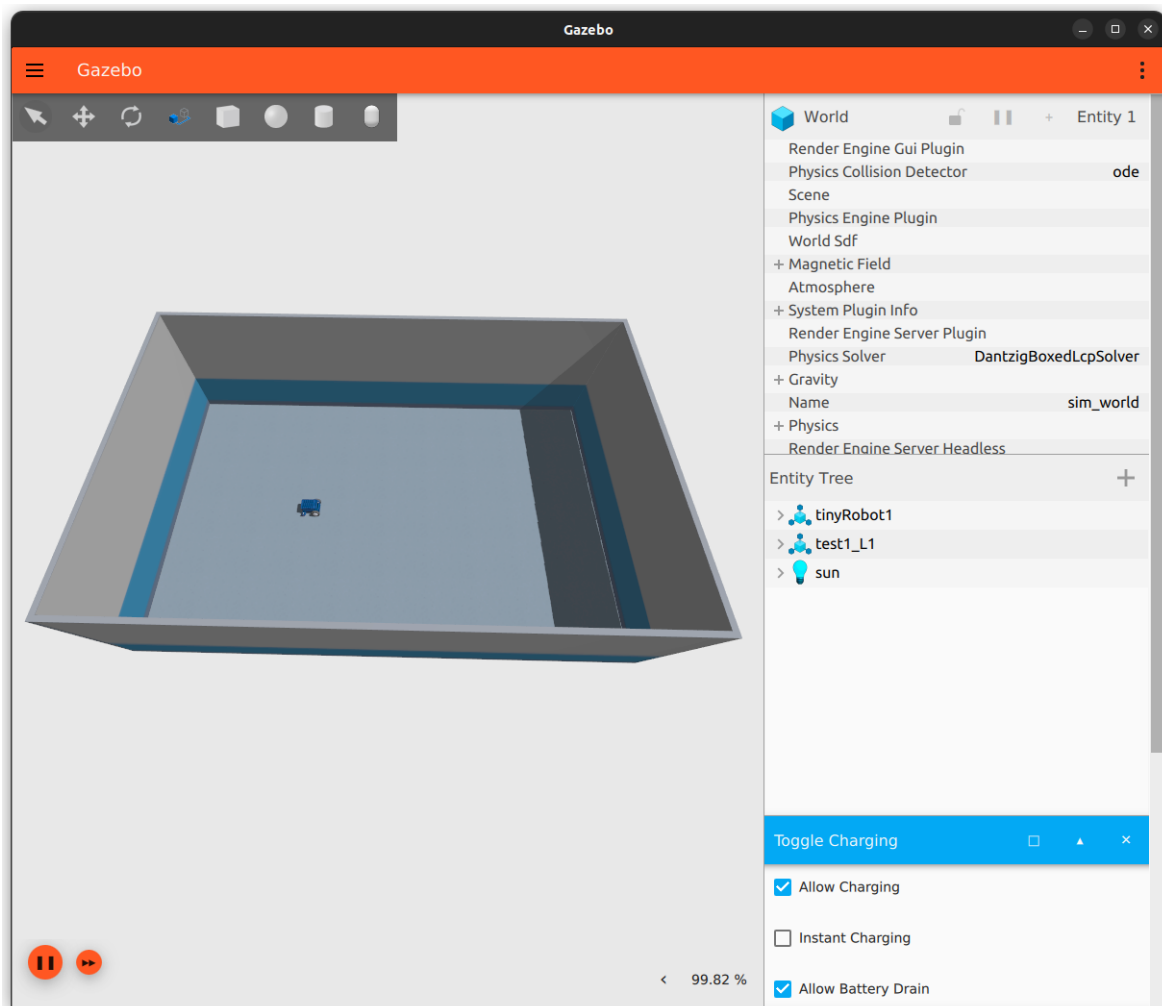


Figure 12: Measurement Simulation

9. Add Doors and Walls:

- Add a door using the door button (or press 'D') and create both ends of the door, the order in which you add the two points for the door decides its orientation but that can be changed modifying the “motion direction” property of the door to -1. Remember to always set a name to your doors, if there is more than one door with an empty name the simulation will crash.

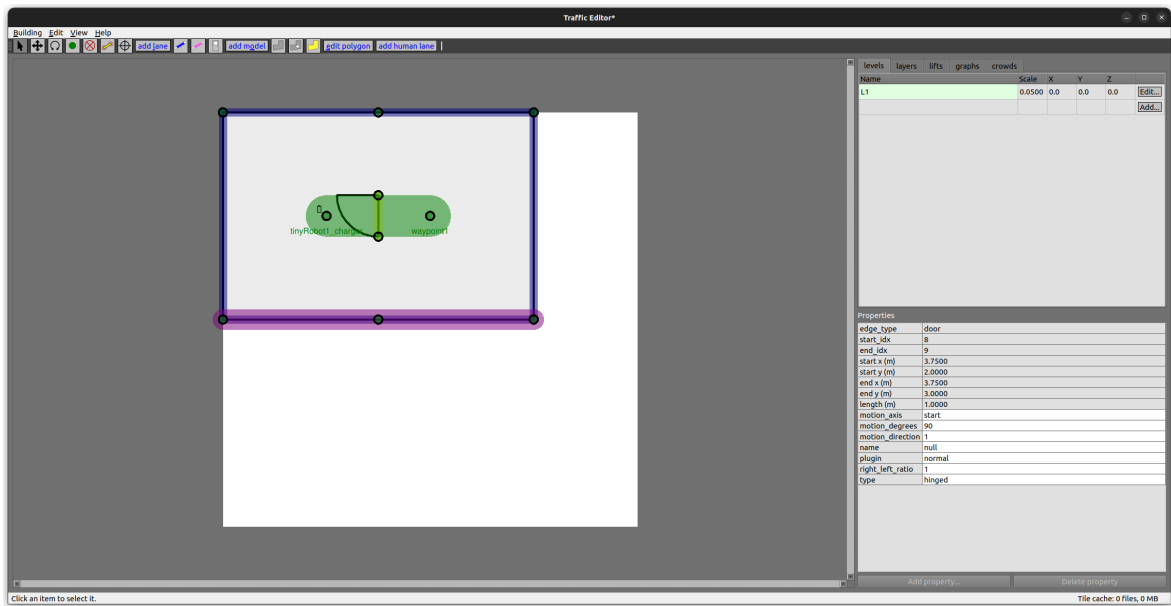


Figure 13: Adding a Door

- Add walls to create two rooms that can be traversed through the door to create a similar scenario to Figure 12.

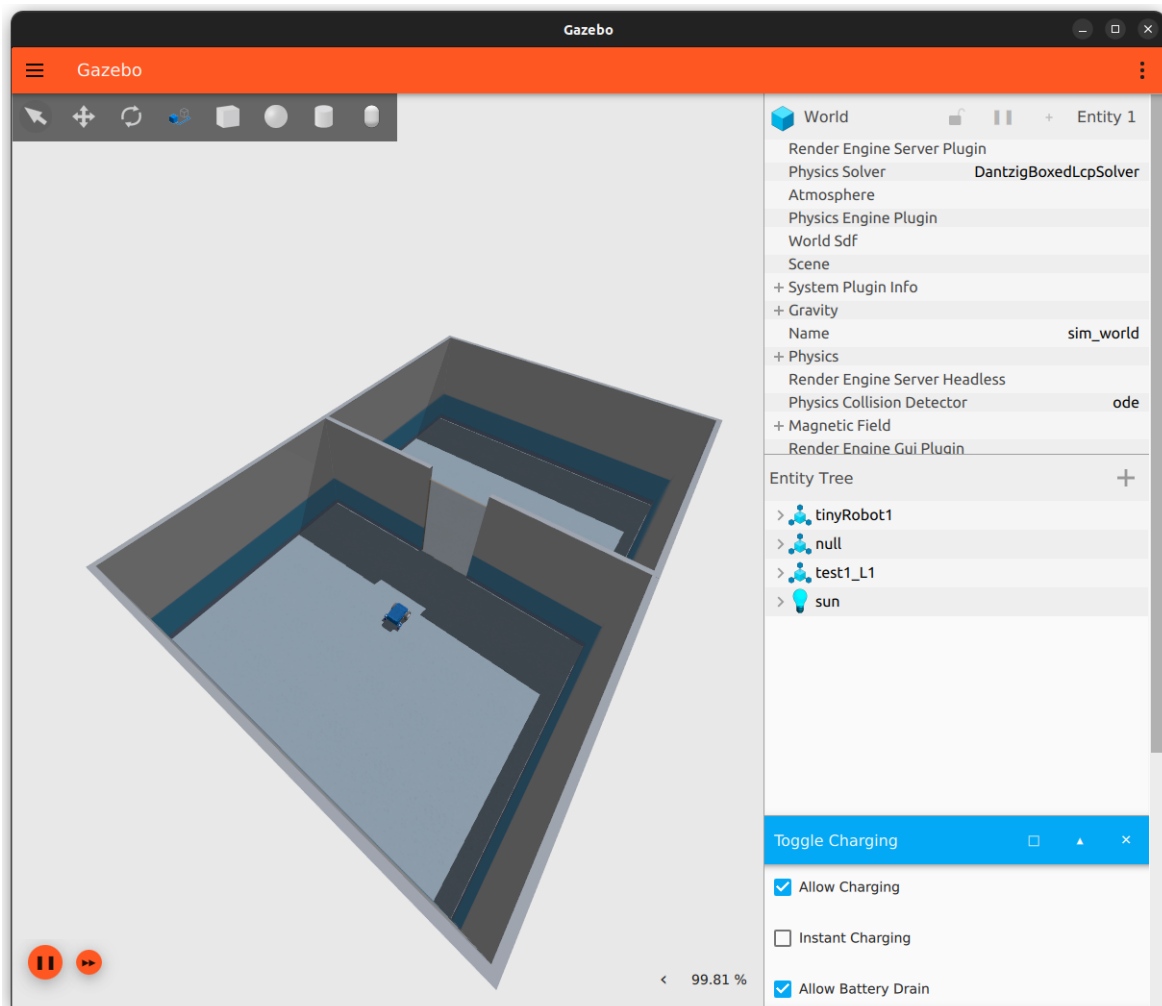


Figure 14: Door Simulation

- Now you can send a task to the robot to move to the waypoint1 with:

```
ros2 run project_tasks dispatch_patrol -n 10 -p waypoint1
    tinyRobot1_charger -F TinyRobot -R tinyRobot1 --use_sim_time
```

- This creates a patrol task that will wander 10 times between the points waypoint1 and tinyRobot1_charger and is assigned to the rinyRobot1 of type TinyRobot, remember to add `--use_sim_time` as this executes the task in simulation time, which is different from the computer time. This can be disabled in the launch files, if you do, remove the `--use_sim_time` option so the task is run in the proper time.

10. Add models to the environment

- You can also add models like furniture, decorations, object and more through the “add model” button or by pressing ‘O’. This will open a pop-up where you can search the item you want to place.

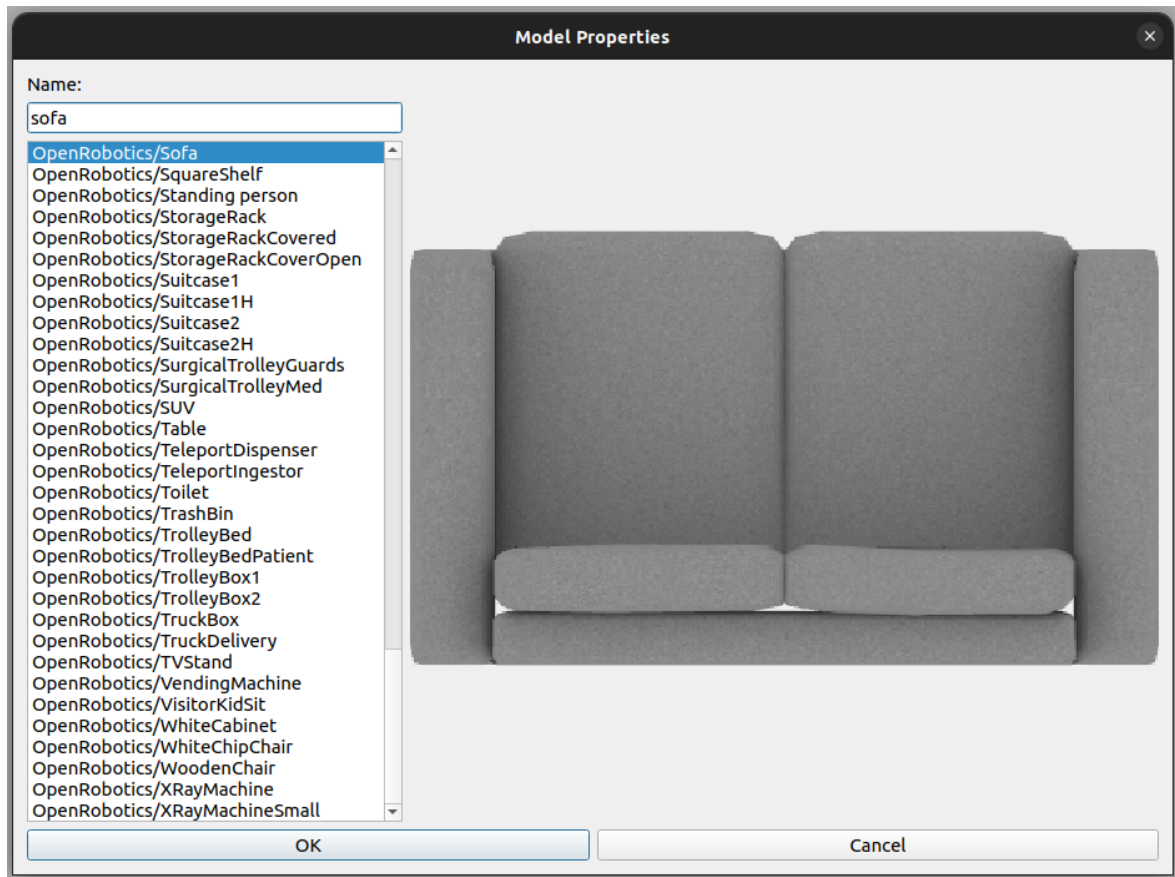


Figure 15: Select a model

- Now you can place it and use the rotate 'R' and move 'M' functionalities to adjust the orientation and position of the object you place. If you place multiple objects they can all be rotated going to 'Edit' -> 'Rotate all models'. rotation.

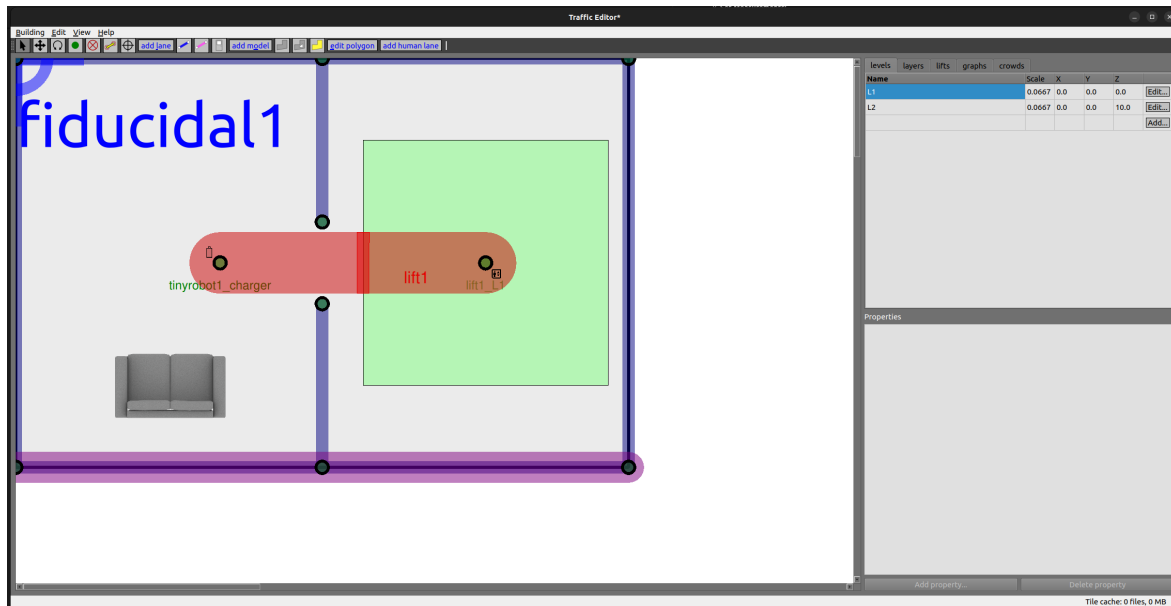


Figure 16: Place models

- Run the simulation and check the results, you should have something similar to Figure 17.

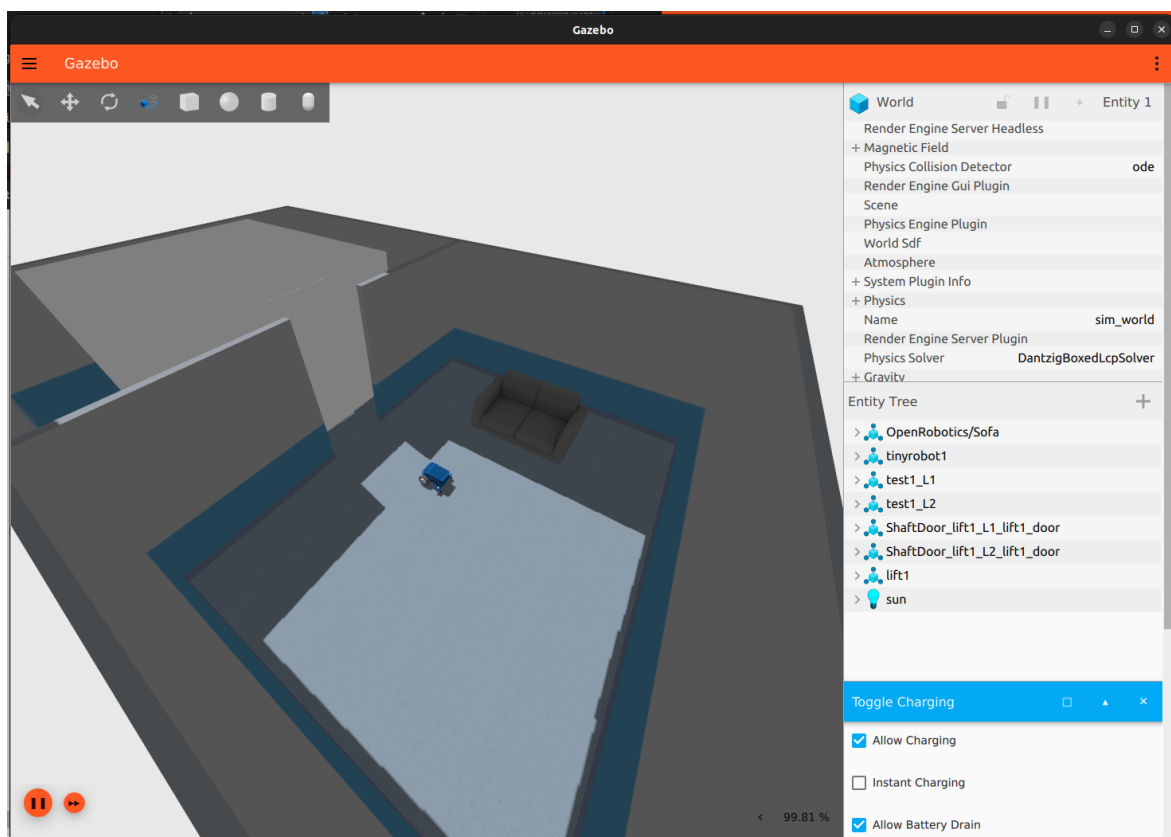


Figure 17: Simulation with models

11. Create Multiple Floors:

- To finish explaining all the tools, we will add a new floor and connect the floors using a lift. In this example, the doors have been removed(Figure 18).

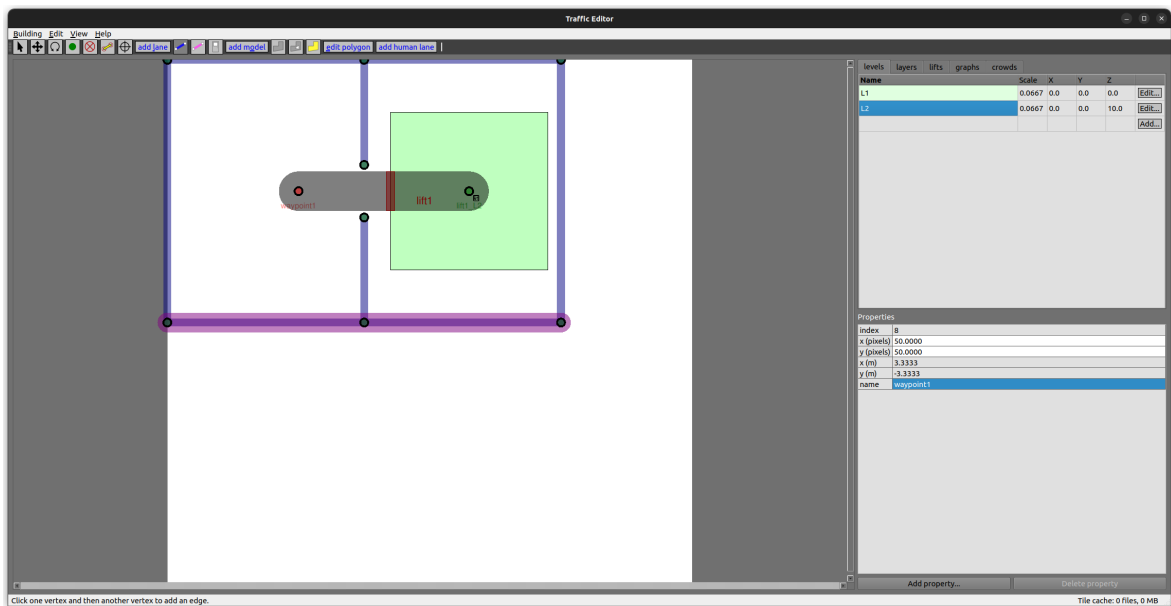


Figure 18: Second Floor

- To create a lift, go to the upper-right panel named lifts and add a new one. Fill in the information such as name, x and y positions, floors, and add one or multiple doors. This is a complex process as you need to align the door to the lift and the map and it may take some trial and error to get the result you want. Once you have the results you want, press “add lift waypoints” as shown in the example so that waypoints for each of the lifts floors are automatically created and then name them, check Figure 19 as an example.
- Try not to move this waypoints because the map generation depends on them, if you move them, click again on “add lift waypoints” to restore them.

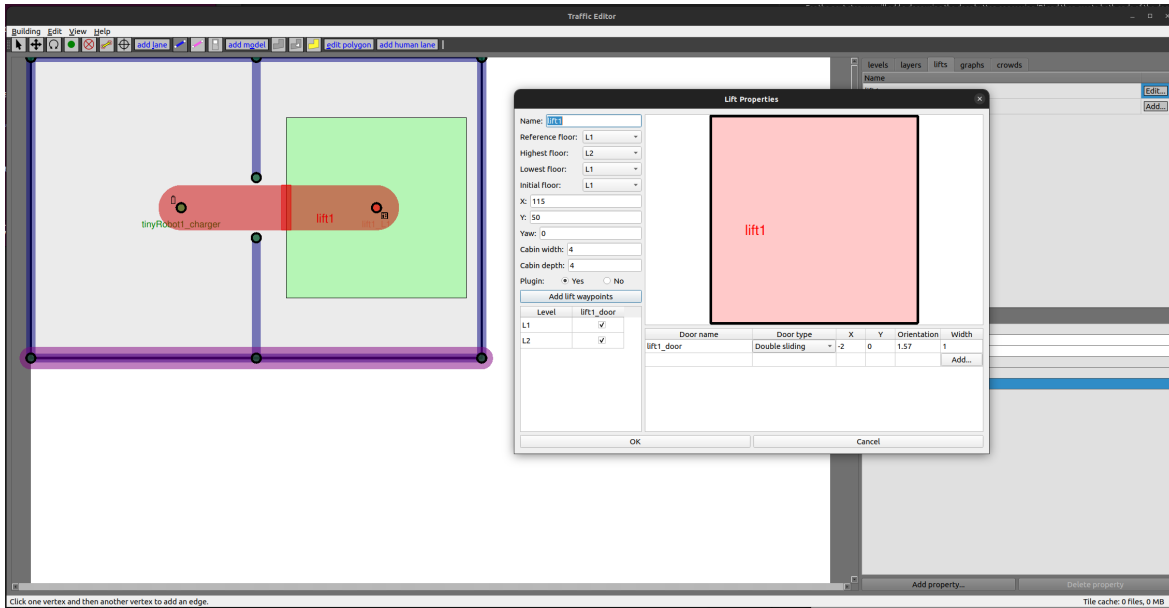


Figure 19: Creating a Lift

5.4. Execute tasks

In order to move the robots we need to execute different tasks, for example, we can order a robot unit to execute a patrol between different waypoints with the command:

```
ros2 run project_tasks dispatch_patrol -n 10 -p waypoint1 waypoint2
waypoint2 -F robot_fleet -R robot_name --use_sim_time
```

In this command we use the “-n” option to repeat the patrol 10 times, the “-p” option to indicate the different waypoints that compose the patrol’s path, the “-F” option to select a fleet of robots that should take this task, the “-R” to name the concrete robot that we want to execute the task with and “--use_sim_time” that, as explained before, launches the task using the simulation time and not the computer time.

There are multiple tasks in the “project_tasks” module like “goto” and “loop”, there is also a task to use teleop and directly control the robot and many others.

Some times it is hard to see what is wrong with the system and why something is not working, for example, when executing the task the request is not processed and we get the message “Did not get a response”. This may be caused by a faulty configuration, always check you are configuring the proper navigation graph, you must also check that you are not misspelling the robot name, the waypoint name, the fleet name or any other attribute of the task. To check if the robot is working you can use the “simple_api_server” that comes with OpenRMF demos, you can access `localhost:8083/robot_list` and this will display all the robots currently working in the system (Figure 20).

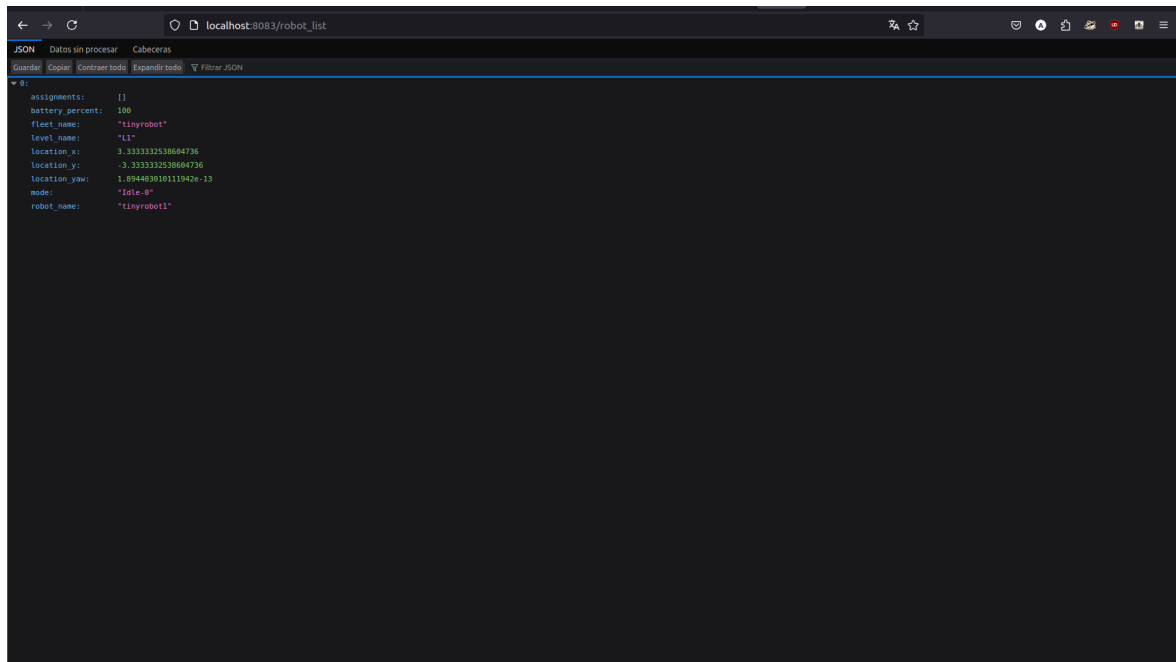


Figure 20: Simple API server displaying a list of robots

I encountered a problem, the task was not executing so I checked the “simple_api_server” and found out that the robot fleet was not able to launch properly since there was no robot listed there, after fixing the nav_graph to point to the proper .yaml file and restarting the simulation I was able to fix it. If you ever encounter this problem check the folder on “build/project_maps/maps/test1/nav_graphs”, the “number.yaml” that appears here should be the one in the “project_config/launch/test1.launch.xml”. If you find multiple number.yaml files it means you created multiple navigation graphs, if this is not intended try to fix it in the “traffic_editor” by deleting the lanes in a different color than the rest.

Now we execute the task again and the robot responds and starts moving as seen in Figure 21.

```

ros2 run project_tasks dispatch_patrol -n 10 -p lift1_L1
tinyrobot1_charger -F tinyrobot -R tinyrobot1 --use_sim_time

```

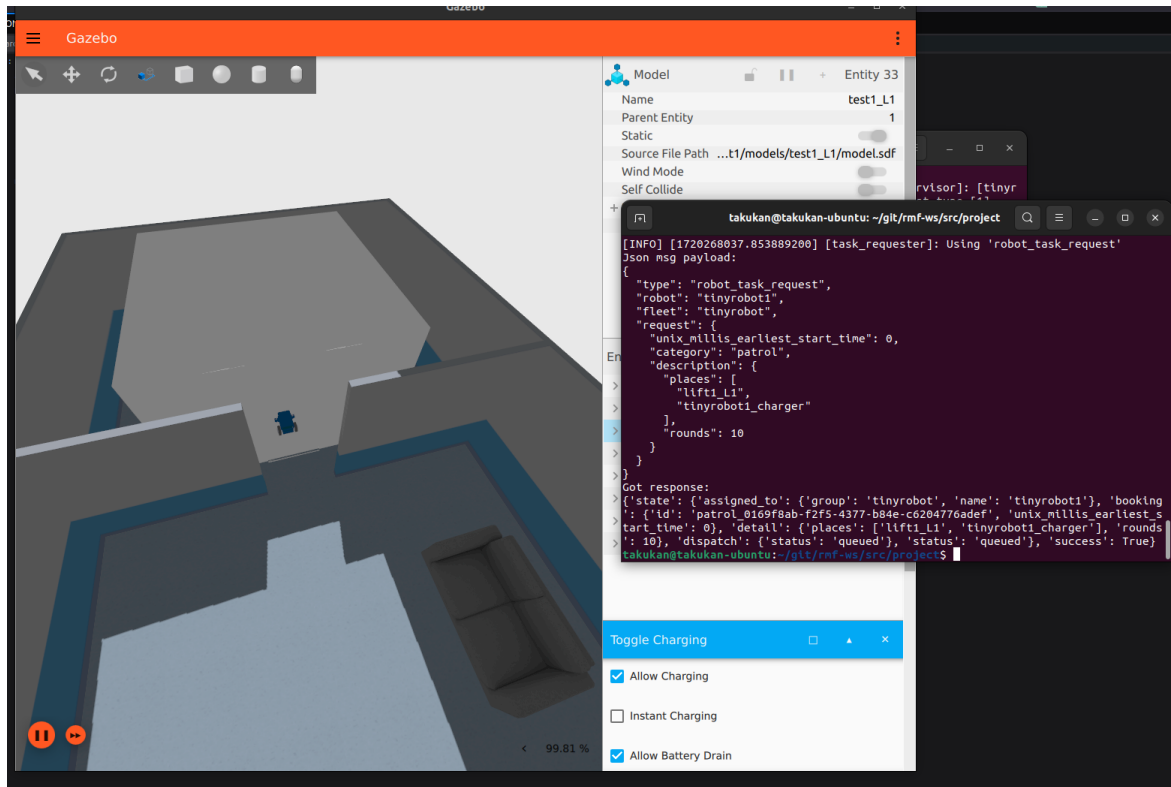


Figure 21: Robot moving after executing a task